

```

.....#..#####.....#####.....
.....#####.....

..#####.....
..#000000000000000000000000#...#####...
..#00000000000#####0000#####000#...
..#0000000000#.....#0000000000000#...
..#0000000000#####0000000000####...
..#####0000000000000000000000#.....
.....#00#####00000#####.....
.....#####.....

```

If you want to see every step of the flood fill algorithm as it fills in the new character, uncomment the `printImage(image)` line ❶ in the `floodFill()` function and run the program again.

The image is represented by a 2D array of string characters. We can pass this `image` data structure, an `x` coordinate and a `y` coordinate, and a new character to the `floodFill()` function. The function notes the character currently at the `x` and `y` coordinates and saves it to the `oldChar` variable ❷.

If the current characters at coordinates `x` and `y` in `image` are not the same as `oldChar`, this is our base case, and the function simply returns. Otherwise, the function continues on to its four recursive cases: passing the `x`- and `y`-coordinates of the bottom ❸, top ❹, right ❺, and left ❻ neighbors of the current coordinates. After these four potential recursive calls are made, the end of the function is an implicit base case, made explicit in our program with a `return` statement ❼.

The flood fill algorithm doesn't have to be recursive. For large images, a recursive function could cause stack overflows. If we were to implement flood fill with a loop and a stack instead, the stack would begin with the `x`- and `y`-coordinates of the starting pixel. The code in the loop would pop the coordinates off the top of the stack, and if that coordinate's pixel matches `oldChar`, it would push the coordinates of the four neighboring pixels. When the stack is empty because the base case is no longer pushing neighbors to the stack, the loop is finished.

However, the flood fill algorithm doesn't necessarily have to use a stack. The pushing and popping of a first-in, last-out stack is effective for backtracking behavior, but the order that the pixels are processed in the flood